

Building Your First Neural Network in PyTorch

A Complete Guide to Training Pipelines from Scratch

PyTorch Tutorial

March 23, 2026

Abstract

This tutorial provides a comprehensive, hands-on introduction to building neural networks in PyTorch from the ground up. We will implement a binary classification model using the Breast Cancer dataset, manually constructing every component—weights, bias, forward pass, loss function, backpropagation, and gradient descent—to understand the underlying mechanics before moving to PyTorch’s built-in modules. By the end, you will understand the complete training pipeline that remains consistent even for complex deep learning models.

Contents

1	Prerequisites and Context	3
1.1	Our Goal	3
1.2	What Makes This Tutorial Different	3
2	The Dataset: Breast Cancer Classification	3
2.1	Dataset Characteristics	3
3	Complete Code Implementation	3
3.1	Step 1: Imports and Data Loading	3
3.2	Step 2: Train-Test Split	4
3.3	Step 3: Feature Scaling	4
3.4	Step 4: Label Encoding	5
3.5	Step 5: Converting to PyTorch Tensors	5
4	Building the Neural Network from Scratch	5
4.1	Understanding the Architecture	5
4.2	Defining the Model Class	6
5	The Training Pipeline	7
5.1	Setting Hyperparameters	7
5.2	Training Loop Implementation	7
5.3	Expected Output	8
5.4	Understanding Critical Details	9
5.4.1	Why <code>torch.no_grad()</code> ?	9
5.4.2	Why <code>zero_grad()</code> ?	9
5.4.3	The Update Rule	9
6	Model Evaluation	9

7	What We've Built: The Big Picture	10
7.1	Training Pipeline Summary	10
7.2	Manual vs. PyTorch Native	10
8	Common Pitfalls and Tips	11
9	Next Steps and Further Learning	11
10	Complete Code Reference	11
11	Conclusion	12

1 Prerequisites and Context

Before beginning this tutorial, you should be familiar with:

- Basic Python programming
- Fundamental concepts of PyTorch tensors (creation, operations, indexing)
- Understanding of computation graphs and automatic differentiation (Autograd)

If you are new to these concepts, it is recommended to review introductory PyTorch materials on tensors and automatic differentiation first.

1.1 Our Goal

Our objective is **not** to achieve state-of-the-art accuracy. Instead, we aim to understand the **workflow** of PyTorch: how data flows through a model, how gradients are computed, and how parameters are updated. We will use a real-world dataset (Breast Cancer) and implement many components manually to see exactly what happens under the hood.

1.2 What Makes This Tutorial Different

- We build a **single-neuron** network (simplest possible)
- We implement **manual elements**: custom weight initialization, custom loss function, manual gradient descent
- The **training pipeline structure** mirrors professional PyTorch code exactly
- We use real data, not toy examples

2 The Dataset: Breast Cancer Classification

We use the Wisconsin Breast Cancer dataset, a classic binary classification problem. The goal is to predict whether a tumor is malignant (M) or benign (B) based on 30 numeric features computed from digitized images of fine needle aspirates (FNA) of breast mass.

2.1 Dataset Characteristics

- **Samples**: 569 patients
- **Features**: 30 numeric attributes (radius, texture, perimeter, area, smoothness, compactness, concavity, etc.)
- **Target**: Diagnosis (M = Malignant, B = Benign)
- **Columns to drop**: `id` (identifier) and `Unnamed: 32` (empty column)

3 Complete Code Implementation

3.1 Step 1: Imports and Data Loading

First, we import necessary libraries and load the data directly from GitHub using pandas:

```

1 import numpy as np
2 import pandas as pd
3 import torch
4 from sklearn.model_selection import train_test_split
5 from sklearn.preprocessing import StandardScaler, LabelEncoder
6
7 # Load dataset
8 df = pd.read_csv('https://raw.githubusercontent.com/gscdit/'
9                  'Breast-Cancer-Detection/refs/heads/master/data.csv')
10
11 print(df.shape) # Output: (569, 33)
12 df.head()

```

Listing 1: Importing Libraries and Loading Data

Key Point: The dataset has 33 columns initially. We must remove `id` and `Unnamed: 32` as they don't contribute to prediction.

```

1 # Drop unnecessary columns
2 df.drop(columns=['id', 'Unnamed: 32'], inplace=True)
3
4 print(df.shape) # Output: (569, 31) - now 30 features + 1 target
5 df.head()

```

Listing 2: Cleaning the Dataset

3.2 Step 2: Train-Test Split

We split the data into training (80%) and testing (20%) sets:

```

1 # Separate features (X) and target (y)
2 # iloc[:, 1:] selects all columns except 'diagnosis' (first column)
3 # iloc[:, 0] selects only the 'diagnosis' column
4
5 X_train, X_test, y_train, y_test = train_test_split(
6     df.iloc[:, 1:],      # Features: all columns except first
7     df.iloc[:, 0],       # Target: first column (diagnosis)
8     test_size=0.2,       # 20% for testing
9     random_state=42      # For reproducibility
10 )
11
12 print(f"Training samples: {X_train.shape[0]}") # 455
13 print(f"Testing samples: {X_test.shape[0]}")  # 114

```

Listing 3: Splitting Data

3.3 Step 3: Feature Scaling

Neural networks perform best when input features are on similar scales. We use `StandardScaler` to transform features to have `mean=0` and `std=1`:

```

1 # Initialize scaler
2 scaler = StandardScaler()
3
4 # Fit on training data and transform
5 X_train = scaler.fit_transform(X_train)
6
7 # Transform test data using training statistics
8 X_test = scaler.transform(X_test)
9
10 # Result: X_train and X_test are now numpy arrays with standardized values

```

```
11 print(X_train[0]) # Example: array([-0.65, -0.15, -0.58, ...])
```

Listing 4: Feature Scaling with StandardScaler

Important: Always fit the scaler on **training data only**, then apply the same transformation to test data. This prevents data leakage.

3.4 Step 4: Label Encoding

Our target labels are currently strings ('M' and 'B'). Neural networks need numeric inputs, so we encode:

- M (Malignant) $\rightarrow$ 1
- B (Benign) $\rightarrow$ 0

```
1 # Initialize encoder
2 encoder = LabelEncoder()
3
4 # Fit and transform training labels
5 y_train = encoder.fit_transform(y_train)
6
7 # Transform test labels
8 y_test = encoder.transform(y_test)
9
10 print(y_train[:10]) # Output: [1 1 0 1 1 0 0 0 0 1]
```

Listing 5: Encoding Labels

3.5 Step 5: Converting to PyTorch Tensors

PyTorch works with **tensors**, not numpy arrays. Conversion is seamless using `torch.from_numpy()`:

```
1 # Convert features to float64 tensors
2 X_train_tensor = torch.from_numpy(X_train)
3 X_test_tensor = torch.from_numpy(X_test)
4
5 # Convert labels to float64 tensors (for mathematical operations)
6 y_train_tensor = torch.from_numpy(y_train)
7 y_test_tensor = torch.from_numpy(y_test)
8
9 print(X_train_tensor.shape) # torch.Size([455, 30])
10 print(y_train_tensor.shape) # torch.Size([455])
11
12 # Verify data type
13 print(X_train_tensor.dtype) # torch.float64
```

Listing 6: Numpy Arrays to PyTorch Tensors

Critical Detail: We use `float64` (double precision) to match the numpy array type. In production, `float32` is standard, but for this educational example, consistency is key.

4 Building the Neural Network from Scratch

4.1 Understanding the Architecture

We build the simplest possible neural network:

- **Input layer:** 30 neurons (one per feature)

- **Output layer:** 1 neuron (binary classification)
- **No hidden layers:** Direct input-to-output connection
- **Activation:** Sigmoid (outputs probability 0-1)

Mathematically: $\hat{y} = \sigma(X \cdot W + b)$

Where:

- X is the input matrix (455×30)
- W is the weight vector (30×1)
- b is the bias scalar (1)
- σ is the sigmoid function

4.2 Defining the Model Class

We use Object-Oriented Programming (OOP) to define our model. This is the standard PyTorch approach:

```

1 class MySimpleNN:
2     def __init__(self, X):
3         """
4         Initialize weights and bias.
5
6         Args:
7             X: Training data tensor (used to determine input size)
8         """
9         # Initialize weights randomly from uniform distribution [0,1)
10        # Shape: (number_of_features, 1) = (30, 1)
11        self.weights = torch.rand(
12            X.shape[1],          # 30 features
13            1,                   # 1 output neuron
14            dtype=torch.float64,  # Match data type
15            requires_grad=True    # Track gradients for backprop!
16        )
17
18        # Initialize bias to zero
19        # Shape: (1,) - scalar bias for single neuron
20        self.bias = torch.zeros(
21            1,                   # Single bias term
22            dtype=torch.float64,
23            requires_grad=True    # Track gradients!
24        )
25
26    def forward(self, X):
27        """
28        Forward pass: compute predictions.
29
30        Args:
31            X: Input features tensor
32
33        Returns:
34            y_pred: Predicted probabilities after sigmoid
35        """
36        # Linear transformation: z = X @ W + b
37        z = torch.matmul(X, self.weights) + self.bias
38
39        # Apply sigmoid activation: y_pred = 1 / (1 + exp(-z))
40        y_pred = torch.sigmoid(z)
41
```

```

42         return y_pred
43
44     def loss_function(self, y_pred, y):
45         """
46         Binary Cross-Entropy Loss (manual implementation).
47
48         Args:
49             y_pred: Predicted probabilities from forward pass
50             y: True labels
51
52         Returns:
53             loss: Scalar loss value
54         """
55         # Clamp predictions to avoid log(0) numerical errors
56         epsilon = 1e-7
57         y_pred = torch.clamp(y_pred, epsilon, 1 - epsilon)
58
59         # Binary Cross Entropy formula:
60         # L = -[y*log(y_pred) + (1-y)*log(1-y_pred)]
61         loss = -(y * torch.log(y_pred) +
62                 (1 - y) * torch.log(1 - y_pred)).mean()
63
64         return loss

```

Listing 7: Defining the Neural Network Class

Key Concepts Explained:

1. **requires_grad=True:** This tells PyTorch to track all operations on these tensors so we can compute gradients later via backpropagation.
2. **Weight Initialization:** We use random values between 0 and 1. Other strategies (Xavier, He initialization) exist but random is fine for this simple case.
3. **Sigmoid Activation:** Squashes any real-valued number to range (0,1), interpretable as probability of class 1.
4. **Loss Clamping:** Prevents taking $\log(0)$ which is undefined (negative infinity).

5 The Training Pipeline

5.1 Setting Hyperparameters

Before training, we define:

```

1 learning_rate = 0.1    # Step size for gradient descent
2 epochs = 25           # Number of complete passes through dataset

```

Listing 8: Hyperparameters

5.2 Training Loop Implementation

The training loop consists of four essential steps, repeated for each epoch:

1. **Forward Pass:** Compute predictions
2. **Loss Calculation:** Measure error
3. **Backward Pass:** Compute gradients via autograd

4. Parameter Update: Adjust weights using gradient descent

```

1 # Create model instance
2 model = MySimpleNN(X_train_tensor)
3
4 print("Initial weights shape:", model.weights.shape) # torch.Size([30, 1])
5 print("Initial bias:", model.bias)                  # tensor([0.])
6
7 # Training loop
8 for epoch in range(epochs):
9     # ===== STEP 1: FORWARD PASS =====
10    # Compute predictions for all training samples
11    y_pred = model.forward(X_train_tensor)
12
13    # ===== STEP 2: LOSS CALCULATION =====
14    # Calculate binary cross-entropy loss
15    loss = model.loss_function(y_pred, y_train_tensor)
16
17    # ===== STEP 3: BACKWARD PASS =====
18    # Compute gradients of loss w.r.t. all parameters
19    # This populates .grad attributes of weights and bias
20    loss.backward()
21
22    # ===== STEP 4: PARAMETER UPDATE =====
23    # Update weights: w_new = w_old - learning_rate * gradient
24    # We use torch.no_grad() because we don't need to track
25    # these operations for gradient computation
26    with torch.no_grad():
27        model.weights -= learning_rate * model.weights.grad
28        model.bias -= learning_rate * model.bias.grad
29
30    # ===== STEP 5: ZERO GRADIENTS =====
31    # CRITICAL: Clear gradients for next iteration
32    # PyTorch accumulates gradients by default!
33    model.weights.grad.zero_()
34    model.bias.grad.zero_()
35
36    # Print progress
37    print(f'Epoch: {epoch + 1}, Loss: {loss.item():.4f}')

```

Listing 9: Complete Training Loop

5.3 Expected Output

You should see the loss decreasing over time:

```

Epoch: 1, Loss: 3.8735
Epoch: 2, Loss: 3.7475
Epoch: 3, Loss: 3.6134
...
Epoch: 23, Loss: 0.7762
Epoch: 24, Loss: 0.7590
Epoch: 25, Loss: 0.7485

```

Why does loss decrease? The model is learning! Gradients point in the direction of increasing loss, so we move parameters in the opposite direction (hence the minus sign in the update rule).

5.4 Understanding Critical Details

5.4.1 Why `torch.no_grad()`?

When updating parameters, we're performing arithmetic on tensors that have `requires_grad=True`. Without `no_grad()`, PyTorch would build a computation graph for these update operations too, wasting memory and potentially causing errors. We only need gradients for the forward/backward pass, not for parameter updates.

5.4.2 Why `zero_grad()`?

PyTorch accumulates gradients by default (useful for certain techniques like gradient accumulation). If we don't zero them, each iteration's gradients add to the previous ones, causing incorrect updates.

5.4.3 The Update Rule

This is vanilla Gradient Descent:

$$W_{new} = W_{old} - \alpha \cdot \frac{\partial L}{\partial W} \quad (1)$$

Where α is the learning rate. The gradient $\frac{\partial L}{\partial W}$ is automatically computed by PyTorch's autograd system when we call `loss.backward()`.

6 Model Evaluation

After training, we evaluate on the test set:

```

1 # Set model to evaluation mode (no gradient tracking needed)
2 with torch.no_grad():
3     # Forward pass on test data
4     y_pred = model.forward(X_test_tensor)
5
6     # Convert probabilities to binary predictions
7     # Threshold: 0.5 (standard for binary classification)
8     # Note: Some experiments use 0.9 for demonstration; 0.5 is typical
9     y_pred_binary = (y_pred > 0.5).float()
10
11     # Calculate accuracy
12     # Compare predictions with true labels, convert to float, take mean
13     accuracy = (y_pred_binary == y_test_tensor).float().mean()
14
15     print(f'Accuracy: {accuracy.item():.4f}')
16
17 # Inspect final parameters
18 print("Trained bias:", model.bias) # Example: tensor([-0.0940])

```

Listing 10: Evaluation Code

Note on Thresholds: While 0.5 is the standard decision boundary for sigmoid outputs, you may experiment with different thresholds (such as 0.9) to observe how they affect accuracy. This is useful for understanding model calibration and decision boundaries.

7 What We've Built: The Big Picture

7.1 Training Pipeline Summary

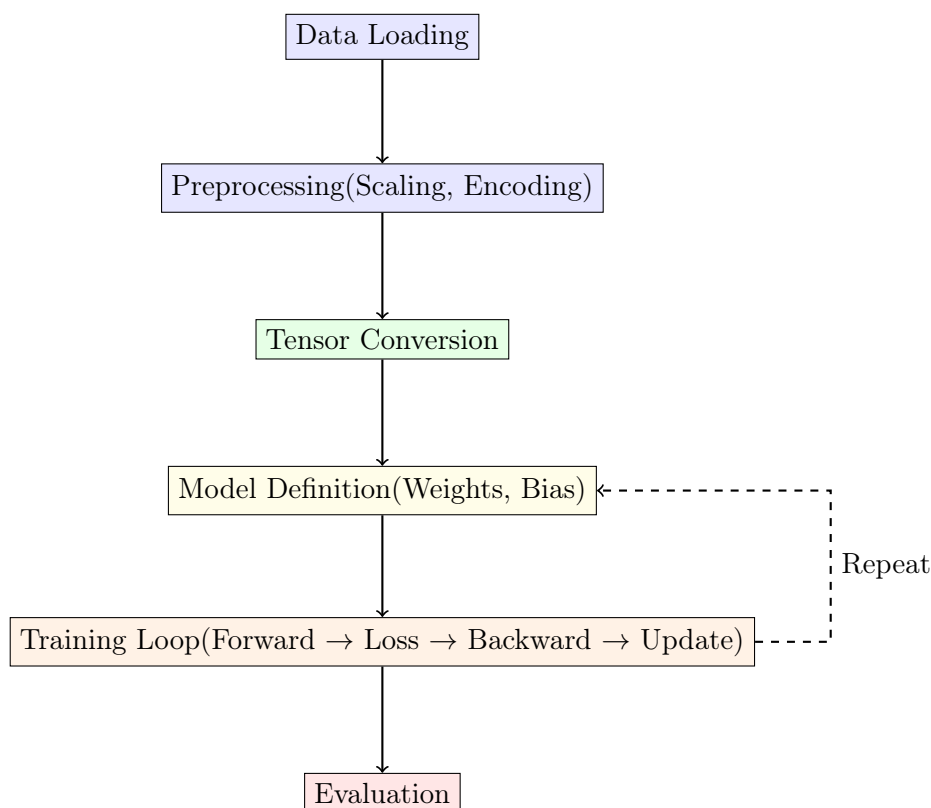


Figure 1: Complete PyTorch Training Pipeline

7.2 Manual vs. PyTorch Native

In this tutorial, we manually implemented:

- Weight and bias initialization
- Forward pass mathematics
- Loss function (Binary Cross Entropy)
- Gradient descent update
- Gradient zeroing

In production PyTorch code, these are replaced by:

- `torch.nn.Module` for model definition
- `torch.nn.Linear` for layers
- `torch.nn.BCELoss` or `BCEWithLogitsLoss`
- `torch.optim.SGD` or `Adam` for optimization
- `optimizer.zero_grad()` and `optimizer.step()`

Why did we do it manually? Understanding what happens under the hood makes you a better practitioner. When models don't train properly, this knowledge helps debug.

8 Common Pitfalls and Tips

1. **Forgetting `zero_grad()`:** Gradients accumulate. Always clear them after each update.
2. **Wrong Data Types:** Ensure tensors match (float64 vs float32). Mismatches cause errors.
3. **Modifying Parameters Without `no_grad()`:** Always wrap parameter updates in `with torch.no_grad():`.
4. **Not Scaling Features:** Neural networks are sensitive to input scales. Always normalize/standardize.
5. **Learning Rate Too High/Low:** If loss doesn't decrease, try adjusting learning rate (0.01 to 0.1 is a good range to test).

9 Next Steps and Further Learning

After mastering this manual implementation, you should explore PyTorch's built-in modules to write cleaner, more efficient code:

- **`nn.Module`:** Proper object-oriented model definition with automatic parameter management
- **`nn.Linear`:** Built-in layer that handles weight initialization and matrix multiplication
- **`nn.BCELoss`:** Optimized, numerically stable loss functions
- **`torch.optim`:** SGD, Adam, RMSprop optimizers with advanced features like momentum and adaptive learning rates
- **`DataLoader`:** Efficient batching, shuffling, and data loading
- **GPU Acceleration:** Moving computations to CUDA devices

The pipeline structure you learned here remains identical—only the internal implementations become more sophisticated and efficient. This foundational knowledge will serve you whether you are building simple logistic regression models or complex deep learning architectures.

10 Complete Code Reference

For convenience, here is the complete, runnable code:

```

1 import numpy as np
2 import pandas as pd
3 import torch
4 from sklearn.model_selection import train_test_split
5 from sklearn.preprocessing import StandardScaler, LabelEncoder
6
7 # Load and preprocess data
8 df = pd.read_csv('https://raw.githubusercontent.com/gscdit/
9                 'Breast-Cancer-Detection/refs/heads/master/data.csv')
10 df.drop(columns=['id', 'Unnamed: 32'], inplace=True)
11
12 X_train, X_test, y_train, y_test = train_test_split(
13     df.iloc[:, 1:], df.iloc[:, 0], test_size=0.2
14 )
15
16 scaler = StandardScaler()
```

```

17 X_train = scaler.fit_transform(X_train)
18 X_test = scaler.transform(X_test)
19
20 encoder = LabelEncoder()
21 y_train = encoder.fit_transform(y_train)
22 y_test = encoder.transform(y_test)
23
24 X_train_tensor = torch.from_numpy(X_train)
25 X_test_tensor = torch.from_numpy(X_test)
26 y_train_tensor = torch.from_numpy(y_train)
27 y_test_tensor = torch.from_numpy(y_test)
28
29 # Define model
30 class MySimpleNN:
31     def __init__(self, X):
32         self.weights = torch.rand(X.shape[1], 1, dtype=torch.float64,
33                                   requires_grad=True)
34         self.bias = torch.zeros(1, dtype=torch.float64, requires_grad=True)
35
36     def forward(self, X):
37         z = torch.matmul(X, self.weights) + self.bias
38         return torch.sigmoid(z)
39
40     def loss_function(self, y_pred, y):
41         epsilon = 1e-7
42         y_pred = torch.clamp(y_pred, epsilon, 1 - epsilon)
43         return -(y * torch.log(y_pred) + (1-y)*torch.log(1-y_pred)).mean()
44
45 # Training
46 learning_rate = 0.1
47 epochs = 25
48 model = MySimpleNN(X_train_tensor)
49
50 for epoch in range(epochs):
51     y_pred = model.forward(X_train_tensor)
52     loss = model.loss_function(y_pred, y_train_tensor)
53     loss.backward()
54
55     with torch.no_grad():
56         model.weights -= learning_rate * model.weights.grad
57         model.bias -= learning_rate * model.bias.grad
58
59     model.weights.grad.zero_()
60     model.bias.grad.zero_()
61     print(f'Epoch: {epoch+1}, Loss: {loss.item():.4f}')
62
63 # Evaluation
64 with torch.no_grad():
65     y_pred = model.forward(X_test_tensor)
66     y_pred_binary = (y_pred > 0.5).float()
67     accuracy = (y_pred_binary == y_test_tensor).float().mean()
68     print(f'Accuracy: {accuracy.item():.4f}')

```

Listing 11: Full Implementation

11 Conclusion

You have now implemented a complete neural network training pipeline in PyTorch from scratch! You understand:

- How data flows from raw CSV to PyTorch tensors

- How to manually initialize and track parameters with `requires_grad`
- The four-step training loop: Forward $\rightarrow$ Loss $\rightarrow$ Backward $\rightarrow$ Update
- The importance of gradient management (`no_grad`, `zero_grad`)
- How to evaluate model performance

This foundation is universal. Whether you're building a simple logistic regression (as we did) or a 100-layer ResNet, the same pipeline applies. Understanding these fundamentals will make you a more effective deep learning practitioner, capable of debugging issues and optimizing training procedures.

Happy Learning!